

Evaluation - Docker - Réseaux - BTSSIO-02 - 25-26

Table des matières

Rappel des étapes Installation VM	1
Étape 1 : Création de l'environnement et des réseaux	2
Étape 2 : Configuration du Pare-feu.....	3
Étape 3 : Mise en place du Serveur Web (DMZ)	4
Étape 4 : Déploiement des Clients	4
Étape 5 : Les Tests (Validation finale)	6

Rappel des étapes Installation VM

Installe VirtualBox sur ton ordinateur comme un logiciel classique.

Lien direct : [Télécharger Ubuntu Desktop \(Site officiel\)](#)

1. Crée une nouvelle VM dans VirtualBox (donne-lui 4 Go de RAM et 25 Go de disque).
2. Insère l'ISO d'Ubuntu (le fichier téléchargé au point 1) quand VirtualBox te demande le disque de démarrage.
3. Laisse-toi guider par l'installateur Ubuntu pour finaliser l'installation.

Étape 1 : Création de l'environnement et des réseaux

- **Explication** : Création du dossier de travail et mise en place de 3 réseaux virtuels Docker isolés représentant nos 3 zones de sécurité (WAN, DMZ, LAN)

```
mkdir dmz-docker-lab && cd dmz-docker-lab
```

```
docker network create --subnet 172.30.0.0/24 wan_net
```

```
docker network create --subnet 172.20.0.0/24 dmz_net
```

```
docker network create --subnet 172.10.0.0/24 lan_net
```

```
Last login: Wed Feb 25 03:19:25 2026 from 192.168.1.181
user-tawabl@user-tawabl-VirtualBox:~$ mkdir dmz-docker-lab
cd dmz-docker-lab
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker network create --subnet 172.30.0.0/24 wan_net
docker network create --subnet 172.20.0.0/24 dmz_net
docker network create --subnet 172.10.0.0/24 lan_net
9a22fff30525693a7f9f16f689952315d68ca22e4cfb8696daa6dee7fe7c6706
b4fb619de08fec3de34c5e2a7327c4bbf4eb20323c9b41fecfec2889fd87544
b869121dea7f917d6850c1408c74bc08357f7190874bad1b9259aa70d7c82436
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
59a63652eaba        bridge             bridge              local
b4fb619de08f        dmz_net            bridge              local
e5a8a2a0629f        host               host                local
b869121dea7f        lan_net            bridge              local
eca3d2970171        none               null                local
9a22fff30525        wan_net            bridge              local
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$
```

- Créer 3 réseaux virtuels isolés avec des adresses IP précises : wan_net, dmz_net, et lan_net.

Vérifier la création : docker network ls

```
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
59a63652eaba        bridge             bridge              local
b4fb619de08f        dmz_net            bridge              local
e5a8a2a0629f        host               host                local
b869121dea7f        lan_net            bridge              local
eca3d2970171        none               null                local
9a22fff30525        wan_net            bridge              local
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$
```

Étape 2 : Configuration du Pare-feu

- Créer un conteneur Alpine nommé firewall connecté au réseau WAN, avec les privilèges administrateur réseau (NET_ADMIN) .

Créer et lancer le Pare-feu :

```
docker run -dit --name firewall \
```

```
--cap-add NET_ADMIN \
```

```
--cap-add NET_RAW \
```

```
--sysctl net.ipv4.ip_forward=1 \
```

```
--network wan_net \
```

Alpine

```
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker run -dit --name firewall \
--cap-add NET_ADMIN \
--cap-add NET_RAW \
--sysctl net.ipv4.ip_forward=1 \
--network wan_net \
alpine
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
```

- Connecter ce pare-feu aux deux autres réseaux (DMZ et LAN) pour qu'il soit au centre de tout .

```
docker network connect dmz_net firewall
```

```
docker network connect lan_net firewall
```

```
Status: Downloaded newer image for alpine:latest
3b7e40db1f39945f8eb103d298682a6976c83898f148b438842201cd40ee09d3
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker network connect dmz_net firewall
docker network connect lan_net firewall
```

- Entrer dans le pare-feu, installer iptables , et taper les règles de sécurité (ex : autoriser le HTTP vers la DMZ, bloquer tout accès du WAN vers le LAN) .

```
docker exec -it firewall sh
```

```
docker exec -it firewall sh
/ # apk update
v3.23.3-190-gc53ea6debd3 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.3-188-g0ddab105b7f [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27573 distinct packages available
```

Étape 3 : Mise en place du Serveur Web (DMZ)

- **Explication** : Lancement du serveur Nginx dans la DMZ . On lui ajoute manuellement les routes de retour pour qu'il sache comment répondre aux clients .

```
docker run -dit --name web_dmz --cap-add NET_ADMIN --network dmz_net nginx
```

```
docker exec -it web_dmz sh
```

```
apt update && apt install -y iproute2
```

```
ip route add 172.10.0.0/24 via 172.20.0.2
```

```
ip route add 172.30.0.0/24 via 172.20.0.2
```

```
exit
```

```
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker run -dit --name web_dmz \
--cap-add NET_ADMIN \
--network dmz_net \
nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
206356c42440: Pull complete
b47f187216b6: Pull complete
1ad233904a11: Pull complete
eedda9fd8786: Pull complete
35ff83c394d6: Pull complete
17d0911eaf62: Pull complete
df0b66c867e4: Pull complete
Digest: sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b3ae1734f3392e
Status: Downloaded newer image for nginx:latest
b6cc61358bfl4ba90d29dc7ff37e24f9689073513957239ba0215e90885f8037e
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$
```

Étape 4 : Déploiement des Clients

- **Explication** : Création des machines de test dans les zones LAN et WAN . On installe curl et on indique à chaque client le chemin (route) vers la DMZ

```
docker run -dit --name lan_client --cap-add NET_ADMIN --network lan_net alpine
```

```
docker exec -it lan_client sh
```

```
apk update && apk add iproute2 curl
```

```
ip route add 172.20.0.0/24 via 172.10.0.2
```

```
exit
```

```
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker run -dit --name lan_client \
--cap-add NET_ADMIN \
--network lan_net \
alpine
1eaa87728aaa01c43adb4a5903ba27cbf6b6f8374556be2542d67e5ce5d8dc65
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$
```

- Installer l'outil curl sur ces deux machines.

```
docker exec -it lan_client sh
```

```
apk update
```

```
apk add iproute2 curl
```

```
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker exec -it lan_client sh
/ # apk update
v3.23.3-190-gc53ea6debd3 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.3-188-g0ddab105b7f [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27573 distinct packages available
/ # apk add iproute2 curl
```

- Leur indiquer la route pour joindre la DMZ.

```
ip route add 172.20.0.0/24 via 172.10.0.2
```

```
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker exec -it web_dmz sh
/ # ip route add 172.20.0.0/24 via 172.30.0.2curl http://172.20.0.3
Error: inet address is expected rather than "172.30.0.2curl".
/ # curl http://172.20.0.3
^C
/ # exit
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker exec -it web_dmz sh
# apt update && apt install -y iproute2
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9670 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [113 kB]
Fetched 10.0 MB in 1s (11.8 MB/s)
All packages are up to date.
Installing:
  iproute2

Installing dependencies:
  libbpf1 libcap2-bin libelf1t64 libmnl0 libpam-cap libtirpc-common libtirpc3t64 libxtables12
```

Étape 5 : Les Tests (Validation finale)

- **Explication** : Validation de l'architecture en effectuant une requête HTTP depuis chaque client pour afficher la page du serveur Nginx .

`docker exec -it lan_client curl http://172.20.0.3`

```
user-tawabl@user-tawabl-VirtualBox:~/dmz-docker-lab$ docker exec -it lan_client curl http://172.20.0.3
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```